生成AIを使った業務の自動化事例紹介

生成AIでソフトウェア開発ワークフローを自律実行する

アジェンダ

- **AIで自動化するアプローチ**: ツール連携・スキル化・オーケストレーションの3つの柱
- **ツール連携**: AIにシステム操作手段を与える
- **現状の業務フロー**: 今回の自動化範囲
- **ワークフローの説明**: 7フェーズの詳細と各スキルの役割
- **スキル・エージェントを育てる**: 継続的改善サイクル

AIで自動化するアプローチ

> **人間がやっていた各ステップをAIが実行できる単位に分解する**

3つの柱

- **ツール連携**: AIにシステムを操作する手段を与える
- **スキル化**: 各作業をAIが実行できる単位に分解
- **オーケストレーション**: スキルをつなげてAIにやり切らせる

AIにシステム操作手段を与える

> ツールがなければAIは外の世界と対話できない

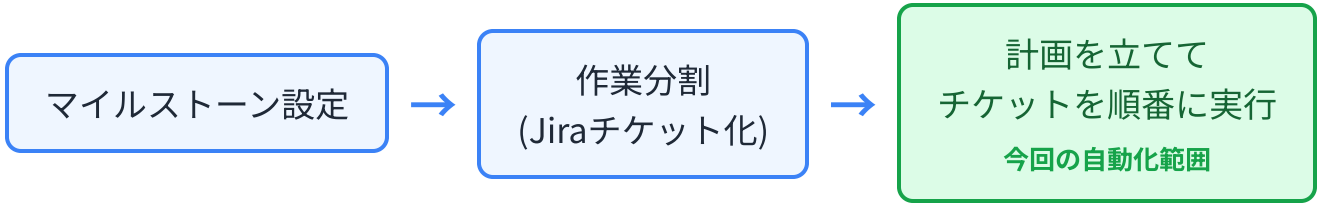
連携システム

システム	操作できること
GitLab	リポジトリ構造把握・Issue取得・MR作成
Jira	チケット取得・進捗更新・ステータス変更
プロジェクトツール	プロジェクト固有ツールの操作

重要な原則

- APIを通じてシステムを操作するスキルを整備する
- スキルは**再利用可能**な単位で設計する

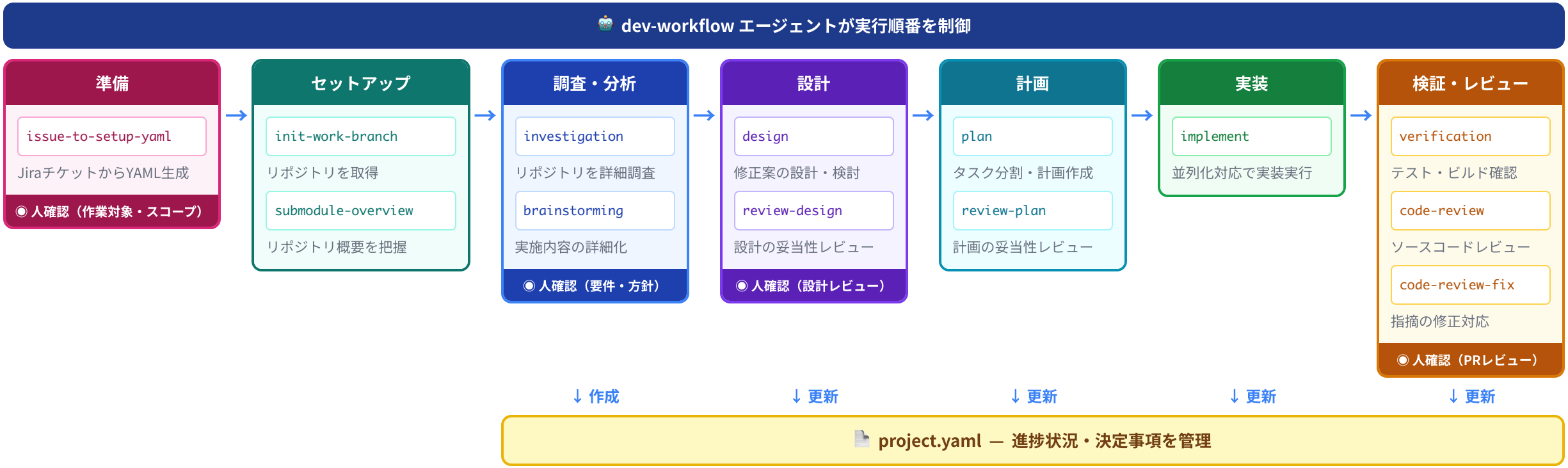
現状の業務フロー



チケット1件の実行フロー

ステップ	内容
① 詳細化	実現したいこと・修正対象の確認と現状把握
② 修正案検討	解決策の検討
③ 修正案レビュー	方針の妥当性確認
④ 実装	コーディング
⑤ 動作確認	テスト・検証
⑥ レビュー → マージ	コードレビュー後に統合

ワークフロー全体像


dev-workflow エージェントが実行順番を制御
準備
issue-to-setup-yaml
JiraチケットからYAML生成
◉ 人確認（作業対象・スコープ）
セットアップ
init-work-branch
リポジトリを取得
submodule-overview
リポジトリ概要を把握
調査・分析
investigation
リポジトリを詳細調査
brainstorming
実施内容の詳細化
◉ 人確認（要件・方針）
設計
design
修正案の設計・検討
review-design
設計の妥当性レビュー
◉ 人確認（設計レビュー）
計画
plan
タスク分割・計画作成
review-plan
計画の妥当性レビュー
実装
implement
並列化対応で実装実行
検証・レビュー
verification
テスト・ビルド確認
code-review
ソースコードレビュー
code-review-fix
指摘の修正対応
◉ 人確認（PRレビュー）
↓ 作成
↓ 更新
↓ 更新
↓ 更新
↓ 更新
project.yaml — 進捗状況・決定事項を管理

フェーズ詳細：準備

Jiraチケットをもとにユーザーと対話しながらゴール・スコープを確定し、AIが実行可能なYAMLに変換する

📥 入力

📋 Jiraチケット

実施内容・背景情報

→

⚙ issue-to-setup-yaml

AIが選択肢を提示

「ゴールは？」A.機能追加 B.バグ修正 C.改善 D.自由入力

「スコープ・制約は？」選択肢 or 自由入力

ユーザーが選択 or 自由入力

「B」（シンプル選択）

「B、でも既存動作は維持して」（自由入力）

「DBマイグレーション不可」（制約を追記）

↻ 対話を繰り返しYAML内容を洗練 → 最終確認で確定

🔗 Jira — チケット情報を取得

👤 **ここが最重要 — AIと対話してゴール・スコープを徹底的に確定する**

→

📤 成果物

setup.yaml

ゴール・スコープ・制約を構造化

フェーズ詳細：セットアップ

作業に必要なリポジトリを取得し、構造を把握してproject.yamlを初期化する

入力

setup.yaml

ゴール・対象リポジトリ情報

→

init-work-branch

・作業ブランチを作成してサブモジュールを初期化
・project.yaml を初期化（以降の進捗管理の起点）

GitLab — ブランチ作成・リポジトリクローン

submodule-overview

・各リポジトリの構造・責務・依存関係を解析
・サブモジュールの役割をドキュメント化

GitLab — ファイル構造・README を参照

→

成果物

作業ブランチ

project.yaml

進捗管理の初期化

フェーズ詳細：調査・分析

修正対象を具体的に特定し、UML図を含む詳細な調査レポートを作成する

入力

- setup.yaml
- リポジトリ

→

brainstorming

AIが確認事項を提示	ユーザーが選択 or 自由入力
「具体的にどんな動作を期待する？」	「〇〇を入力したら△△になること」
「実装アプローチはどれが近い？」選択肢 or 自由入力	「Bパターン、ただし既存APIは維持」
「テストはどの粒度まで？」単体/結合/E2E	「単体+E2Eで十分」
「考慮すべきリスク・制約は？」	「後方互換必須」

↻ 対話で要件・アプローチ・テスト戦略・受け入れ条件を確定

investigation

・対象コードの詳細調査・影響範囲のマッピング
・UML図（クラス図・シーケンス図）を自動生成

GitLab — コード参照・ファイル解析

→

成果物

- 調査レポート
- UML図
- project.yaml 更新

フェーズ詳細：設計

修正案を設計し、2つのAIモデルをサブエージェントとして使用してレビューする

📥 入力

- 調査レポート
- project.yaml

↻ 差し戻し時に再実行するループ区間

⚙ design

・API仕様・データ構造・処理フローを設計
・テスト計画を含む詳細設計書を作成

⚙ review-design

・複数モデルで設計をレビューして指摘事項を統合

🤖 サブエージェント: Claude Opus 4.6 Codex → 結果を統合

⚙ create-mr-pr DRモード

・設計内容をドラフトMRとして作成・人間にレビューを依頼

🦊 GitLab — ドラフトMRを作成

👤 人間レビュー — GitLabのドラフトMRで設計を確認

✅ 承認
→ 次フェーズ（計画）へ

↩ 差し戻し
→ design に戻りループ区間を再実行

📤 成果物

- 詳細設計書
- ドラフトMR

フェーズ詳細：計画

設計を実装タスクに分割し、2つのAIモデルをサブエージェントとして使用してレビューする

📥 入力

- 詳細設計書
- project.yaml

→

↻ 指摘がなくなるまでループ

⚙ plan

・タスクを細かく分割して依存関係を整理
・並列実行可能なタスクを識別・E2Eテスト計画を作成

⚙ review-plan

・複数モデルで実装計画をレビューして指摘事項を統合

🤖 サブエージェント: Claude Opus 4.6 Codex → 結果を統合

→

📤 成果物

- タスクリスト
- E2Eテスト計画
- project.yaml 更新

フェーズ詳細：実装

独立タスクをサブエージェントで並列実行してTDDで品質を確保し、テストで検証する

入力

- タスクリスト
- project.yaml

↻ テスト失敗時に再実行するループ

implement

・TDD: 失敗テストを先に書いてから実装コードを書く

サブエージェント: 独立タスクを 並列実行

GitLab — 実装コードを Push・コミット管理

verification

・テスト / ビルド / リント / E2E を実行して検証

・受け入れ条件との照合 → 不合格なら implement に戻る

成果物

- 実装コード
- テスト結果
- project.yaml 更新

フェーズ詳細：検証・レビュー

2つのAIモデルをサブエージェントとして使用してコードをレビューし、MRを完成させる

📥 入力

- 実装コード
- project.yaml

→

⚙ create-mr-pr Codeモード

・ドラフトMRを作成してレビュー用の場所を用意

🦊 GitLab — ドラフトMRを作成

↻ 指摘がなくなるまでループ

⚙ code-review

・チェックリストベースで精査 → MRにレビュー結果を記載

🤖 サブエージェント: Claude Opus 4.6 Codex → 結果を統合

⚙ code-review-fix

・指摘事項を修正 → 再レビューへ

ドラフト解除

・指摘なしで完了 → ドラフトを解除して人間レビューへ

🦊 GitLab — ドラフト解除・MRを正式化

🔗 Jira — チケットを完了ステータスに更新

👤 人間レビュー — GitLabのMRで最終コードを確認

✅ 承認
→ MRをマージ

↩ 差し戻し
→ code-review-fix に戻りループを再実行

→

📤 成果物

- 正式MR
- 完了 Jiraチケット

スキル・エージェントを育てる

> うまくいかなかった部分はセッションを振り返って改善に反映

継続的改善サイクル



ポイント

- ユーザーの指摘はすぐにスキルへフィードバック
- プロジェクト固有の知識をスキルに蓄積していく

まとめ

生成AI自動化の3ステップ

1. **連携**: AIにシステム操作の手段（ツール）を与える
2. **分解**: 業務をAIが実行できる単位（スキル）に分解する
3. **統合**: スキルをオーケストレーションして自律実行させる

成功の鍵

> **スキルとエージェントを活用しながら**
> **プロジェクト固有の知識を蓄えて育てていくことが大事！**

ありがとうございました

生成AIと人間が協調して、より良い開発サイクルを実現しましょう